

Речь по слайдам к защите дипломного проекта

Смирнова Екатерина Сергеевна

Тема: «Разработка информационной системы мониторинга и анализа погодных условий для авиационных рейсов»

Как читать	Идти по порядку слайдов. Блок «Говорить» можно читать почти дословно.
Темп	Спокойно, без спешки. Теоретическая часть — 5-7 минут, демонстрация — отдельно.
Важно	На демонстрации говорить не все подряд, а показывать ключевой сценарий: рейс, риск, метеоролог, история.

Основная речь по слайдам

Слайд 1. Титульный лист

На слайде	Тема дипломного проекта, колледж, руководитель, автор.
Переход	Далее я кратко обосную, почему эта тема актуальна.

Говорить

Здравствуйте, уважаемые члены комиссии. Меня зовут Смирнова Екатерина Сергеевна. Тема моего дипломного проекта: «Разработка информационной системы мониторинга и анализа погодных условий для авиационных рейсов».

В рамках работы была разработана веб-система, которая помогает диспетчеру оценивать погодные риски по рейсам, получать рекомендации и взаимодействовать с метеорологом.

Слайд 2. Актуальность, новизна, значимость

На слайде	Погода как причина задержек и инцидентов, новизна расчета риска по этапам полета.
Переход	После актуальности перехожу к цели и задачам проекта.

Говорить

Актуальность проекта связана с тем, что погодные условия являются одной из важных причин задержек и потенциально опасных ситуаций в авиации. Сильный ветер, низкая видимость, гроза, обледенение и осадки могут повлиять на возможность выполнения рейса.

Новизна моей работы заключается в том, что риск оценивается не одним общим числом, а по этапам полета: отдельно для вылета, маршрута и прибытия. Это дает диспетчеру более понятную картину, где именно возникает проблема.

Практическая значимость состоит в поддержке принятия решений: система собирает данные в одном интерфейсе, показывает факторы риска и сохраняет историю действий.

Слайд 3. Цель и задачи

На слайде	Цель разработки и список задач проекта.
Переход	Теперь покажу, на каких технологиях построена система.

Говорить

Целью проекта была разработка информационной системы мониторинга и анализа погодных условий для авиационных рейсов. Система должна автоматизировать получение метеоданных, выполнять расчет уровня риска и выдавать диспетчеру рекомендации.

Для достижения цели я проанализировала предметную область, спроектировала архитектуру и алгоритм расчета риска, разработала клиентскую и серверную части, спроектировала хранение данных и историю изменений.

Также была реализована резервная работа при отказе внешнего погодного API и проведена проверка основных пользовательских сценариев.

Слайд 4. Техничко-экономическое обоснование

На слайде	Java, Spring Boot, React, PostgreSQL, Docker.
Переход	Далее перехожу к общей архитектуре приложения.

Говорить

Для серверной части выбран Java и Spring Boot. Это надежный стек для корпоративных информационных систем, он хорошо подходит для REST API, работы с базой данных и интеграций.

Клиентская часть разработана на React. Он позволяет создать интерактивный интерфейс диспетчера: формы, карту, таблицу рейсов, фильтры, уведомления и модальные окна.

Для хранения используется PostgreSQL. В базе сохраняются рейсы, решения диспетчера, запросы метеорологу, ответы и история операций.

Docker Compose используется для развертывания, чтобы frontend, backend и база данных запускались одинаково на локальной машине и на сервере.

Слайд 5. Технологическая часть: архитектура

На слайде	Схема клиентской части, backend, базы данных и внешних сервисов.
Переход	Теперь подробнее расскажу про главный алгоритм — расчет риска.

Говорить

Архитектура системы разделена на три основных слоя. Первый слой — frontend, где работают интерфейсы диспетчера и метеоролога. Второй слой — backend на Spring Boot, который содержит бизнес-логику. Третий слой — PostgreSQL, где сохраняются данные.

Frontend обращается к backend через REST API. Для оперативных обновлений используется WebSocket: например, при изменении риска, создании рейса или ответе метеоролога интерфейс может обновляться без ручной перезагрузки.

Погодные данные поступают через внешний погодный API, а при его недоступности система использует резервный сценарий.

Слайд 6. Алгоритм риска и рекомендации диспетчеру

На слайде	Формулы риска аэропорта, маршрута, итогового риска и шкала рекомендаций.
Переход	После алгоритма важно показать, какие данные хранятся в системе.

Говорить

В системе риск считается по трем направлениям: риск вылета, риск прибытия и маршрутный риск. Для аэропортов учитываются ветер, порывы, видимость, давление, температура, облачность, осадки и опасные явления.

Маршрутный риск учитывает расстояние между аэропортами, географические особенности, ветровой фон и перепад давления. Расстояние рассчитывается по координатам аэропортов.

Итоговый риск вычисляется как взвешенная оценка: 40 процентов — вылет, 40 процентов — прибытие и 20 процентов — маршрут. После этого риск попадает в шкалу от низкого до критического.

На основе итогового риска система формирует рекомендацию: APPROVE — рейс возможен, DELAY — рекомендуется задержка, CANCEL — выполнение не рекомендуется, REQUEST_METEO — требуется уточнение данных у метеоролога.

Слайд 7. Конструкторская часть: структура данных

На слайде	Схема данных: рейсы, маршруты, метеоданные, аэропорты, история, пользователи.
Переход	Дальше расскажу, как система ведет себя при сбое погодного API.

Говорить

В базе данных основная сущность — рейс. Для рейса хранятся номер, тип воздушного судна, аэропорт вылета, аэропорт назначения, время вылета и прибытия, рассчитанные риски и рекомендация.

Отдельно сохраняется история операций. Это нужно, чтобы можно было посмотреть, когда риск пересчитывался, каким было старое и новое значение, какое решение принял диспетчер.

Запросы метеорологу и ответы также сохраняются. При этом старые ответы не удаляются, потому что для диспетчерской системы важен аудит: нужно видеть, на основании каких данных принималось решение.

Часть данных хранится как JSON-снимки. Это сделано осознанно: такие снимки фиксируют состояние ответа метеоролога и факторов риска именно на момент операции.

Слайд 8. Отказоустойчивость

На слайде	Сценарий: OpenWeather, сбой, кэш, fallback, запрос метеорологу, перерасчет риска.
Переход	Теперь кратко об экономической части проекта.

Говорить

Важная часть проекта — отказоустойчивость. Если внешний погодный API недоступен, система не должна полностью останавливать работу диспетчера.

Сначала выполняется запрос к погодному сервису по координатам аэропорта. Если сервис недоступен, используется резервный механизм: локальный кэш или синтетическая fallback-модель.

В интерфейсе диспетчер видит предупреждение, что параметры рассчитаны по резервному источнику. Это важно для прозрачности: пользователь понимает, что данные не являются обычным ответом API.

Если данных недостаточно, диспетчер может отправить запрос метеорологу. Метеоролог заполняет METAR, TAF и дополнительные параметры, после чего backend пересчитывает риск и сохраняет изменения в истории.

Слайд 9. Экономическая часть

На слайде	Эффективность диспетчерской деятельности за счет оперативной оценки рисков.
Переход	После экономической части подведу ожидаемые результаты разработки.

Говорить

Экономическая значимость проекта заключается в сокращении времени на анализ погодной обстановки и подготовку решения по рейсу.

Вместо ручного сравнения разрозненных данных диспетчер получает единую оценку риска, список факторов и рекомендацию системы.

Это снижает нагрузку на специалиста, уменьшает вероятность ошибки и повышает прозрачность принятия решений.

Слайд 10. Ожидаемые результаты

На слайде	Автоматизация риска, поддержка решений, надежная работа при отсутствии данных, история.
Переход	Далее укажу основные источники, на которые опиралась разработка.

Говорить

В результате проекта реализована автоматизация оценки риска рейса по этапам полета. Система рассчитывает риск на основе погодных параметров и характеристик маршрута.

Диспетчер получает рекомендации по рейсу и видит причины, которые повлияли на риск.

При отсутствии внешних данных система продолжает работу через кэш, fallback-модель и запрос к метеорологу.

Также реализована история изменений, где фиксируются пересчеты риска, погодные данные и решения диспетчера.

Слайд 11. Основные источники информации

На слайде	ГОСТы, документация Leaflet, OpenWeatherMap, PostgreSQL, React, Spring Boot.
-----------	--

Переход	После теоретической части я перехожу к демонстрации работающей системы.
---------	---

Говорить

При выполнении проекта использовались нормативные и технические источники. В частности, ГОСТы по мониторингу опасных метеорологических явлений, автоматизированным метеорологическим системам и авиационным рискам.

Также использовалась документация Spring Boot, React, PostgreSQL, Leaflet и OpenWeatherMap.

Эти источники помогли обосновать выбор технологий, структуру системы и параметры, которые учитываются при оценке погодного риска.

Слайд 12. Переход к демонстрации

На слайде	Фраза о переходе к работающему интерфейсу.
Переход	Если демонстрация завершена, можно перейти к финальному слайду с благодарностью.

Говорить

Дальше я переключаюсь на работающий интерфейс системы и показываю полный пользовательский сценарий.

Сначала я создам рейс, выберу город и аэропорт вылета, затем аэропорт назначения и время вылета. После этого система построит маршрут на карте и рассчитает риск.

Затем я покажу таблицу рейсов, фильтры, историю, запрос метеорологу, ответ метеоролога и перерасчет риска после получения новых данных.

Слайд 13. Спасибо за внимание

На слайде	Завершение и готовность ответить на вопросы.
Переход	Если комиссия попросит схемы, можно перейти к приложениям.

Говорить

На этом основная часть защиты завершена. В результате был разработан рабочий прототип информационной системы мониторинга и анализа погодных условий для авиационных рейсов.

Система включает frontend, backend, базу данных, расчет риска, WebSocket-обновления, работу с метеорологом, историю операций и Docker-развертывание.

Спасибо за внимание. Я готова ответить на ваши вопросы.

Слайд 14. Функциональная схема веб-системы

На слайде	Приложение: функциональная схема.
Переход	Следующий слайд показывает роли пользователей и варианты использования.

Говорить

На этой схеме показаны основные функциональные блоки системы: диспетчерский интерфейс, метеорологический интерфейс, серверная логика, база данных и внешний погодный сервис.

Схема показывает, как пользовательские действия переходят в обработку на backend и далее сохраняются в базе данных.

Слайд 15. Диаграмма вариантов использования

На слайде	Приложение: use case диаграмма.
Переход	Дальше показан процесс создания рейса.

Говорить

На диаграмме вариантов использования показаны основные роли: диспетчер и метеоролог.

Диспетчер создает рейсы, просматривает карту, анализирует риск, принимает решение и отправляет запросы. Метеоролог получает запрос, заполняет данные и отправляет ответ диспетчеру.

Слайд 16. BPMN-схема создания рейса

На слайде	Приложение: процесс создания рейса.
Переход	Следующая схема относится к запросу метеорологу.

Говорить

BPMN-схема показывает последовательность создания рейса: ввод данных, проверка формы, получение погоды, расчет риска, сохранение рейса и отображение результата диспетчеру.

Если какие-то данные введены некорректно, система не дает создать рейс и показывает сообщение об ошибке.

Слайд 17. BPMN-схема запроса данных у метеоролога

На слайде	Приложение: процесс запроса и ответа метеоролога.
Переход	Последняя схема показывает перерасчет риска

	после получения данных.
--	-------------------------

Говорить

Здесь показан процесс взаимодействия диспетчера и метеоролога. Диспетчер выбирает рейс и отправляет запрос на уточнение погодных данных.

Метеоролог получает уведомление, заполняет обязательные поля, проходит валидацию формата и отправляет данные обратно диспетчеру.

Если метеоролог открыл запрос, но не отправил ответ, запрос остается непрочитанным и не считается закрытым.

Слайд 18. Схема перерасчета риска рейса

На слайде	Приложение: процесс перерасчета риска.
Переход	На этом можно завершить ответы по приложениям.

Говорить

Эта схема показывает, что после ответа метеоролога система извлекает данные из METAR и TAF, учитывает дополнительные параметры и пересчитывает риск рейса.

Новый риск сохраняется на backend, отображается диспетчеру и фиксируется в истории изменений.

Таким образом, решение диспетчера принимается уже с учетом уточненной метеорологической информации.

Короткие ответы на возможные вопросы

Почему WebSocket?

WebSocket нужен для оперативных обновлений без ручной перезагрузки страницы. Если меняется риск, создается рейс или приходит ответ метеоролога, диспетчер видит изменения сразу.

Почему часть данных хранится как JSON?

Для истории важно сохранять не только отдельные поля, но и полный снимок состояния на момент решения: факторы риска, набор запрошенных данных и ответ метеоролога. Это помогает аудиту и восстановлению контекста.

Что такое METAR и TAF?

METAR — фактическая авиационная метеосводка, TAF — прогноз. Из них система извлекает ветер, видимость, облачность, давление и другие параметры для перерасчета риска.

Что происходит при отказе погодного API?

Система использует резервный источник и показывает диспетчеру предупреждение. Если данных недостаточно, можно отправить запрос метеорологу и после ответа пересчитать риск.

Как принимается решение по рейсу?

Решение принимает диспетчер, а система дает рекомендацию на основе рассчитанного риска и факторов: разрешить, задержать, отменить или запросить метеоданные.

Финальная фраза, если нужно закончить быстрее

В результате проекта была разработана работающая информационная система, которая объединяет создание рейсов, получение погоды, расчет риска, взаимодействие с метеорологом, историю решений и Docker-развертывание. Система помогает диспетчеру принимать решение быстрее и прозрачнее.